

程序设计 II 第 8 周 题目汇总（完整版）

来源：Matrix · 课程「程序设计 II（25 级 曾坤 计算机学院）」含：完整题面、提交文件、平台提供的初始文件（.hpp/main.cpp 等）。选择题含标准答案。

共 7 项

25 程设II-周 8-课后 1

类型：编程题

题面

实现矩阵类（深拷贝）

题目

实现一个矩阵类 Matrix，使用动态二维数组存储矩阵数据。要求实现：

1. 构造函数
2. 拷贝构造函数（实现深拷贝）
3. 析构函数释放动态内存
4. 输入函数：读取矩阵元素
5. 成员函数 reverse()：将矩阵元素按 行优先顺序展开后反转
6. print 函数：按矩阵格式输出元素

输入

n m
n × m 个整数

输出

拷贝构造生成 B 反转并用 B 初始化的矩阵 C

输入样例

```
2 2
1 2
3 4
```

输出样例

```
1 2
3 4
4 3
2 1
```

提交文件：matrix.cpp

平台提供的初始文件

文件 main.cpp：

```
#include<iostream>
#include "matrix.h"

using namespace std;
```

```
int main(){

    int n,m;

    cin>>n>>m;

    Matrix A(n,m);

    A.input();

    Matrix B=A;

    B.print();

    B.reverse();

    Matrix C(B);

    C.print();
}
```

文件 matrix.h：

```
#ifndef MATRIX_H
#define MATRIX_H

#include<iostream>
using namespace std;

class Matrix{

private:

    int **a;
    int n,m;

public:
```

```
Matrix(int n,int m);

Matrix(const Matrix &other);

~Matrix();

void input();

void reverse();

void print();
};

#endif
```

25 程设II-周 8-课后 2

类型：编程题

题面

题目名称：分析浅拷贝的错误并修正赋值运算符

题目描述：下面给出了一个 Person 类，它含有一个 char* name 指针，并在堆上动态分配内存。如果只使用编译器提供的默认赋值运算符（浅拷贝），当执行 p1 = p2; 时，p1 和 p2 的 name 指针将指向同一块内存。当这两个对象销毁时，同一块内存会被 delete[] 两次，从而引发段错误（Double Free），同时 p1 原有的内存也会泄漏。

请完成以下任务：在 class.cpp 中补全赋值运算符 operator= 的实现，使其执行**深拷贝**，并正确处理自赋值情况（Self-assignment），解决上述内存安全问题。

输入格式：输入在一行中包含两个不含空格的字符串，分别代表 p1 和 p2 的初始名字，由空格分隔。

输出格式：输出共两行，分别代表执行 p1 = p2; 后，p1 和 p2 的打印结果。

输入示例：

Alice Bob

输出示例：

Bob
Bob

提交文件：class.cpp

平台提供的初始文件

文件 class.hpp：

```

#ifndef CLASS_HPP
#define CLASS_HPP

class Person {
private:
    char* name;
public:
    Person(const char* n);
    ~Person();
    Person& operator=(const Person& other);
    void print() const;
};

#endif

```

文件 main.cpp :

```

#include <iostream>
#include <cstring>
#include "class.hpp"
using namespace std;

int main() {
    char name1[100], name2[100];
    if (!(cin >> name1 >> name2)) return 0;

    Person p1(name1);
    Person p2(name2);

    p1 = p2;

    p1.print();
    p2.print();

    return 0;
}

```

25 程设II-周 8-课后 3

类型：编程题

题面

实现动态字符串列表类

题目

实现动态字符串列表类 `StringList`，用于存储多个字符串。类内部使用 **动态内存 (new)** 管理字符串数组。

要求：

- 1 构造函数 `StringList(int n)` 创建可存储 n 个字符串的列表。
- 2 成员函数 `void set(int index, const string& s)` 设置第 $index$ 个字符串 ($index$ 从 0 开始)。
- 3 运算符重载函数 `string& operator[](int index)` 使对象可以像数组一样访问：

```
A[0] = "apple";  
cout << A[0];
```
- 4 `const` 成员函数 `void print() const` 按顺序输出所有字符串，字符串之间用空格分隔。
- 5 拷贝构造函数 `StringList(const StringList& other)` 实现深拷贝
- 6 赋值运算符重载 `StringList& operator=(const StringList& other)` 实现深拷贝
- 7 析构函数：释放动态内存。

输入格式

整数 `n`
`n` 个字符串

输出格式

分别调用拷贝构造和赋值运算符的结果：

B:
字符串列表
C:
字符串列表

输入样例

3
apple banana orange

输出样例

B:
apple banana orange

C:
apple banana orange

提交文件：class.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include <string>
#include "class.hpp"

using namespace std;

int main() {

    int n;
    cin >> n;

    StringList A(n);

    for (int i = 0; i < n; i++) {

        string s;
        cin >> s;

        A.set(i, s);
    }

    // 调用拷贝构造
    StringList B = A;

    // 调用赋值运算符
    StringList C(n);
    C = A;

    // 使用运算符重载
    if(n > 0)
        A[0] = "changed";

    cout << "B:" << endl;
    B.print();

    cout << "C:" << endl;
```

```
    C.print();  
}
```

文件 class.hpp :

```
#ifndef CLASS_HPP  
#define CLASS_HPP
```

```
#include <iostream>  
#include <string>
```

```
using namespace std;
```

```
class StringList {
```

```
private:
```

```
    string *data;  
    int size;
```

```
public:
```

```
    // 构造函数
```

```
    StringList(int n);
```

```
    // 拷贝构造函数
```

```
    StringList(const StringList& other);
```

```
    // 赋值运算符
```

```
    StringList& operator=(const StringList& other);
```

```
    // 运算符重载
```

```
    string& operator[](int index);
```

```
    // 设置字符串
```

```
    void set(int index, const string& s);
```

```
    // const 成员函数
```

```
    void print() const;
```

```
    // 析构函数
```

```
    ~StringList();
```

```
};
```

```
#endif
```

25 程设II-周 8-课后 4

类型：编程题

题面

题目名称：实现计数器类（前置/后置自增/自减运算符重载）

题目描述：实现一个 Counter 类，模拟一个整型计数器。要求实现：

1. 构造函数 Counter(int val = 0) 初始化计数器值。
2. 前置自增 operator++() 和后置自增 operator++(int)
3. 前置自减 operator--() 和后置自减 operator--(int)
4. int get() const 返回当前值
5. void print() const 输出当前值并换行

输入格式：一个整数 n，表示计数器的初始值。

输出格式：依次输出以下操作后的值（每个结果占一行）：

1. 前置自增后的值
2. 后置自增前的值（即原值）
3. 后置自增后的实际值
4. 前置自减后的值
5. 后置自减前的值（即原值）
6. 后置自减后的实际值

输入示例：

5

输出示例：

6
6
7
6
6
5

提交文件：class.cpp

平台提供的初始文件

文件 class.hpp：

```
#ifndef CLASS_HPP
#define CLASS_HPP
```

```
class Counter {
private:
```



```

    int value;
public:
    Counter(int val = 0);
    Counter& operator++();
    Counter operator++(int);
    Counter& operator--();
    Counter operator--(int);
    int get() const;
    void print() const;
};

#endif

```

文件 main.cpp :

```

#include <iostream>
#include "class.hpp"
using namespace std;

int main() {
    int n;
    if (!(cin >> n)) return 0;

    Counter c(n);

    cout << (++c).get() << endl;
    cout << (c++).get() << endl;
    cout << c.get() << endl;

    cout << (--c).get() << endl;
    cout << (c--).get() << endl;
    cout << c.get() << endl;

    return 0;
}

```

25 程设II-周 8-课堂 1

类型：编程题

题面

实现深拷贝字符串类

题目

实现字符串类 MyString，要求实现以下成员：

- 构造函数
- 拷贝构造函数（实现深拷贝）
- 赋值运算符重载（实现深拷贝）
- 析构函数
- print 函数

输入两个字符串；输出通过拷贝构造函数和赋值运算符重载赋值后的三个 MyString 对象的字符串内容；

输入示例

一行输入两个字符串，如：

hello world

字符串不包含空格。

输出示例

输出三个字符串，每个占一行。如：

world

world

hello

提交文件：class.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include <cstring>
#include "class.hpp"
```

```
using namespace std;
```

```
int main(){

    string a,b;
    cin>>a>>b;

    MyString s1(a.c_str());
    MyString s2(b.c_str());

    MyString s3 = s1;

    s1 = s2;

    s1.print();
    s2.print();
```

```
s3.print();  
}
```

文件 class.hpp :

```
#ifndef CLASS_HPP  
#define CLASS_HPP
```

```
#include <iostream>  
#include <cstring>
```

```
using namespace std;
```

```
class MyString{
```

```
private:
```

```
    char *str;
```

```
public:
```

```
    MyString(const char *s="");
```

```
    MyString(const MyString &other);
```

```
    MyString& operator=(const MyString &other);
```

```
    ~MyString();
```

```
    void print();
```

```
};
```

```
#endif
```

25 程设II-周 8-课堂 2

类型：编程题

题面

题目名称：动态整型数组类 IntArray 的深拷贝实现

题目描述：实现一个 IntArray 类，用于管理动态整型数组。要求实现以下功能：

1. 构造函数 IntArray(int n, int initVal = 0)：创建长度为 n 的数组，所有元素初始化为 initVal。
2. 构造函数 IntArray(int n, int* initArr)：创建长度为 n 的数组，并使用传入的指针数组 initArr 进行深拷贝初始化。
3. 拷贝构造函数：实现深拷贝。
4. 赋值运算符重载：实现深拷贝，需正确处理自赋值情况。

- 析构函数：释放动态分配的内存。
- 成员函数 `void print() const`：输出数组所有元素，元素间用空格分隔，行末换行。

输入格式： 第一行一个整数 `n`，表示数组长度。第二行 `n` 个整数，作为数组的初始化值。

输出格式： 输出共三行，顺序如下：

- 第一行：通过拷贝构造函数创建的对象的内容。
- 第二行：通过赋值运算符赋值的对象的内容。
- 第三行：赋值操作后的原对象的内容。

输入示例：

```
5
1 3 5 7 9
```

输出示例：

```
1 3 5 7 9
1 3 5 7 9
1 3 5 7 9
```

提交文件： `class.cpp`

平台提供的初始文件

文件 `main.cpp`：

```
#include <iostream>
#include "class.hpp"
using namespace std;

int main() {
    int n;
    if (!(cin >> n)) return 0;

    int* arr = new int[n];
    for (int i = 0; i < n; ++i) {
        cin >> arr[i];
    }

    IntArray a(n, arr);
    delete[] arr;

    IntArray b = a;

    IntArray c(n, 0);
    c = a;

    b.print();
```

```

    c.print();
    a.print();

    return 0;
}

文件 class.hpp :

#ifndef CLASS_HPP
#define CLASS_HPP

class IntArray {
private:
    int* data;
    int size;
public:
    IntArray(int n, int initVal = 0);
    IntArray(int n, int* initArr);
    IntArray(const IntArray& other);
    IntArray& operator=(const IntArray& other);
    ~IntArray();
    void print() const;
};

#endif

```

25 程设II-周 8-理论题

类型：选择题（20 题）

第 1 题

在 C++ 中，运算符重载本质上是：

- A. 宏定义替换
- B. 一种特殊的函数调用
- C. 编译器的内置指令
- D. 对运算符优先级的修改

答案：B

第 2 题

下列关于 C++ 运算符重载的说法，正确的是：

- A. 可以创建新的运算符，如 **
- B. 可以改变运算符的优先级和结合性
- C. 运算符重载不能改变运算符的操作数个数

- D. 所有的运算符都可以被重载

答案：C

第 3 题

默认的拷贝构造函数执行的是哪种复制策略？

- A. 深拷贝 (Deep Copy)
- B. 浅拷贝 (Shallow Copy)
- C. 引用计数拷贝
- D. 写时复制 (Copy-on-Write)

答案：B

第 4 题

在成员函数重载双目运算符时，左操作数是通过什么隐式传递的？

- A. 第一个参数
- B. this 指针
- C. 全局变量
- D. 引用

答案：B

第 5 题

下列可以作为单目运算符使用的是：

- A. %
- B. =
- C. ++
- D. []

答案：C

第 6 题

将运算符重载为类的成员函数时，以下说法正确的是：

- A. 左操作数必须是该类的对象或其派生类对象
- B. 右操作数必须是基本数据类型
- C. 无法访问类的私有成员
- D. 必须声明为 static

答案：A

第 7 题

在重载 `operator+` 时，若将第二个参数声明为 `const Integer& Int`，这样做的主要目的是：

- A. 允许传入右值（如 `a + 3`）
- B. 防止函数内部意外修改 `Int` 对象，并避免不必要的拷贝
- C. 允许修改 `Int` 对象的值
- D. 提高运算符的优先级

答案：B

第 8 题

若 `operator+` 为成员函数，则 `a.operator+(b)` 等价于？

- A. `operator+(a, b)`
- B. `a + b`
- C. `b + a`
- D. `a.operator+(a, b)`

答案：B

第 9 题

下列哪种情况**必须**使用友元函数（或非成员函数）来重载运算符？

- A. 重载 `=` 运算符
- B. 重载 `<<` 运算符用于输出自定义对象
- C. 重载 `++` 运算符
- D. 重载 `-` 运算符

答案：B

第 10 题

在重载双目运算符的成员函数声明中，末尾加上 `const`（如 `T operator+(const T2& b) const;`），其作用是：

- A. 保证函数内部不修改成员变量
- B. 允许该函数被常量对象调用
- C. 保证右操作数不被修改
- D. 将返回值声明为常量

答案：B

第 11 题

在 C++ 中，前置自增运算符（例如 `++a`）典型的类内重载函数声明是哪一个？

- A. T& operator++();
- B. T operator++(int);
- C. void operator++();
- D. T& operator+(int);

答案：A

第 12 题

重载后置自增运算符（如 a++）时，参数列表中必须带有一个 int 类型的虚拟形参。这个形参的实际作用是什么？

- A. 用于指定自增操作的步长大小。
- B. 仅仅是一个语法约定，用于向编译器区分当前重载的是后置版本而非前置版本。
- C. 用于保存自增前对象的初始值。
- D. 接收操作数的内存地址。

答案：B

第 13 题

为什么前置自增（++a）通常返回左值引用（T&），而后置自增（a++）通常返回右值副本（T）？

- A. 因为 C++ 编译器强制要求必须这么写，否则会报语法错误。
- B. 前置运算需要返回修改后的对象本身；后置运算需要“先用后增”，必须返回一份自增前的历史状态副本。
- C. 返回引用会占用更多的内存空间，后置自增为了优化性能所以返回值。
- D. 两者其实都可以返回引用，只是习惯问题。

答案：B

第 14 题

如果开发者没有在类中显式重载赋值运算符（operator=），C++ 编译器会自动提供一个缺省的赋值运算符。这种缺省操作属于哪种拷贝？

- A. 深拷贝（Deep Copy）
- B. 移动赋值（Move Assignment）
- C. 浅拷贝（Shallow Copy）
- D. 零拷贝（Zero Copy）

答案：C

第 15 题

当类的成员变量包含指针（并在堆上动态分配了内存）时，如果使用缺省的赋值运算符（浅拷贝）执行 a = b;，会导致什么最严重的后果？

- A. 编译无法通过，提示禁止给指针赋值。
- B. a 和 b 的指针内容会各自独立，互不影响。
- C. 会导致 a 原有的内存泄漏，且析构时同一块内存会被释放两次（Double Free），引发段错误。
- D. 会导致程序运行速度大幅度变慢。

答案：C

第 16 题

规范的重载复制赋值运算符（operator=）通常在函数体的最开始执行一项至关重要的判断，这项判断是？

- A. 检查系统内存是否足够分配
- B. 检查形参对象是否为空。
- C. 检查是否为自赋值（即 if (this != &other)）。
- D. 检查形参对象的引用计数是否大于零。

答案：C

第 17 题

根据标准规范，实现“深拷贝”赋值运算符时，操作指针成员的正确步骤顺序是什么（假设非自赋值且空间需重分配）？

- A. 先分配新存储 new int[]，再 delete[] 旧存储，最后执行复制。
- B. 先执行数据复制，再 delete[] 原有存储。
- C. 直接通过 new int[] 重新覆盖原指针，不进行 delete 操作以提高性能。
- D. 先 delete[] 原有存储，然后根据传入对象的大小分配新存储，最后执行内存复制操作。

答案：D

第 18 题

用户自定义的复制赋值运算符通常会将返回值类型设计为对类本身的引用（T&），并返回 *this。这样设计的核心目的是什么？

- A. 允许连续赋值（例如 a = b = c;）。
- B. 避免函数返回时调用一次析构函数
- C. 告诉编译器此时执行的是深拷贝。
- D. 防止外部代码随意修改对象的私有成员。

答案：A

第 19 题

在实现复合运算赋值（如 +=）时，课件提到了一种实现思路：先调用已重载的加法（operator+），再将结果赋给自身（*this = this->operator+(other);）。这种方法的特点是？

- A. 它比直接修改成员变量（如 x += other.x）更加高效，是唯一推荐做法。
- B. 虽然代码得到了复用，但会因为产生临时对象和额外的拷贝赋值导致效率低于直接修改自身成员。
- C. 它是实现复合赋值唯一正确的方法。
- D. 这种写法在语法上是不被 C++ 编译器允许的。

答案：B

第 20 题

假设类 IntArray 有指针成员 int *a。当使用深拷贝策略实现了 operator= 后，执行 a = b;，请问执行完毕后 a 对象的指针（a.a）和 b 对象的指针（b.a）有什么联系？

- A. a.a 和 b.a 指向同一个内存地址（共用一处堆空间）。
- B. a.a 的指针会变成 nullptr，而数据全部转移给 b.a。
- C. b.a 会被释放，a.a 独占数据
- D. a.a 和 b.a 指向完全不同的内存地址，但这两块内存中存储的数据内容是相同的

答案：D
